



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 3

Integrante(s): 120002850 323064039

323071378 323214636

426059576 426058744

Semestre: 2027-1

Fecha de entrega: 11-Septiembre-2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	3
2.1. HashMap	3
2.2. TreeSet	3
2.3. Palíndromos y posiciones decimales	3
2.4. Validación de entrada	3
2.5. Métodos estáticos	4
3. Desarrollo	5
3.1. Investigación de colecciones y aplicaciones	5
3.2. Organización de la aplicación	6
3.3. Lectura y comprobación del intervalo	8
3.4. Comparación de posiciones simétricas.....	11
4. Resultados	13
4.1. Clasificación de entradas válidas.....	13
4.2. Rechazo de valores fuera del intervalo.....	14
4.3. Rechazo de datos no admitidos como int.....	15
5. Conclusión	17
6. Referencias	18

1. Introducción

1.1. Planteamiento del problema

Se requiere investigar el funcionamiento de las colecciones HashMap y TreeSet, así como presentar tres aplicaciones de cada una. Además, se solicita desarrollar en Java una aplicación que permita leer un entero positivo de cinco dígitos y determinar si es un palíndromo tomando en cuenta las siguientes restricciones:

- Ante un valor fuera del intervalo permitido, se debe mostrar un error y solicitar otro dato.
- Para organizar la solución se exige el uso de métodos estáticos.

1.2. Motivación

Mediante la comparación de colecciones se puede fundamentar la elección de una estructura según se requiera asociar claves con valores o mantener elementos únicos en orden. Con la comprobación de palíndromos se ejercita la extracción de dígitos, la evaluación de condiciones y la repetición controlada. Al validar la entrada antes de procesarla, se evita aplicar la comprobación a valores que no satisfacen el dominio establecido.

1.3. Objetivos

Explicar las características de ambas colecciones e identificar tres usos concretos de cada una. Para el segundo ejercicio, se busca validar el tipo y el intervalo del dato, comparar sus posiciones simétricas mediante operaciones enteras y comunicar el resultado. Asimismo, se propone separar la lectura y la comprobación en métodos estáticos y verificar su comportamiento con entradas válidas, límites del intervalo y datos incorrectos.

2. Marco Teórico

2.1. HashMap

Con HashMap se guardan datos en pares de clave y valor. Se utiliza la clave como identificador para consultar el valor asociado; por ejemplo, se puede relacionar un número de cuenta con el nombre de un alumno. Internamente, se realiza un cálculo a partir de la clave para localizar los datos con rapidez. No se permiten claves repetidas: al guardar otro valor con la misma clave, se reemplaza el anterior. Tampoco se garantiza que los datos se recorran en el orden en que se agregaron [1].

2.2. TreeSet

Mediante TreeSet se almacenan elementos ordenados y sin duplicados. Para mantener ese orden se utiliza una estructura de árbol y se comparan los elementos al agregarlos. Por ejemplo, al ingresar los números 5, 2 y 5, se conservan únicamente 2 y 5, en ese orden. Se puede usar el orden habitual de los datos, como el orden numérico, o establecer un criterio propio. Además de agregar y eliminar elementos, se puede comprobar si un dato ya se encuentra en el conjunto [2].

2.3. Palíndromos y posiciones decimales

Se llama palíndromo a una secuencia que se lee igual de izquierda a derecha que de derecha a izquierda. Para comprobar un número de cinco dígitos, se compara el primero con el quinto y el segundo con el cuarto; no se necesita comparar el dígito central. Para separar los dígitos se utilizan la división entera, con la que se obtiene el cociente sin decimales, y el residuo, con el que se obtiene lo que sobra de una división. Por ejemplo, al dividir 12321 entre 10000 se obtiene 1 como cociente entero, y al dividirlo entre 10 se obtiene 1 como residuo [3].

2.4. Validación de entrada

Se valida la entrada para comprobar que el dato ingresado cumple las condiciones del programa antes de utilizarlo. Con Scanner se puede verificar si se admite como un entero de tipo int; de lo contrario, se descarta el dato y se solicita uno nuevo [4]. Después se comprueba que el número esté entre 10000 y 99999, incluidos ambos límites. Si no se cumple esta condición, se muestra un mensaje de error y se vuelve a pedir el número hasta recibir un valor válido.

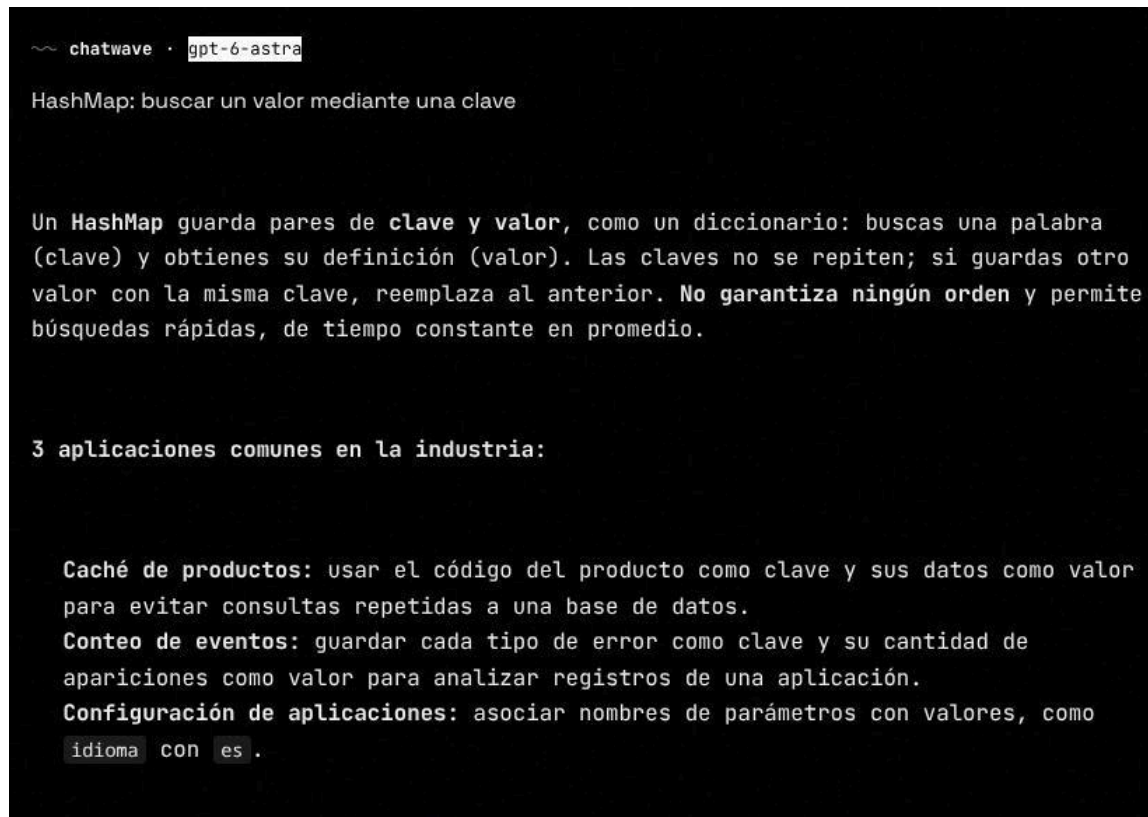
2.5. Métodos estáticos

Con un método se agrupan instrucciones para realizar una tarea específica, como leer un número o comprobar si es un palíndromo. Al declararlo estático, se puede llamar desde la clase sin crear un objeto de ella. Se pueden proporcionar datos al método y recibir un resultado al terminar su tarea [5]. En esta práctica se utilizan métodos estáticos para separar la lectura, la comprobación y la coordinación del programa. Así se facilita la comprensión del código y la revisión de cada parte.

3. Desarrollo

3.1. Investigación de colecciones y aplicaciones

Para el primer ejercicio se contrastaron las propiedades de las colecciones con la documentación oficial de Java [1, 2]. Se empleó el nombre `TreeSet` para la colección indicada como «`ThreeSet`» en el enunciado. Se plantearon los siguientes ejemplos ilustrativos, sin atribuir su implementación al programa de palíndromos.



```
~ chatwave · gpt-6-astra

HashMap: buscar un valor mediante una clave

Un HashMap guarda pares de clave y valor, como un diccionario: buscas una palabra (clave) y obtienes su definición (valor). Las claves no se repiten; si guardas otro valor con la misma clave, reemplaza al anterior. No garantiza ningún orden y permite búsquedas rápidas, de tiempo constante en promedio.

3 aplicaciones comunes en la industria:

Caché de productos: usar el código del producto como clave y sus datos como valor para evitar consultas repetidas a una base de datos.
Conteo de eventos: guardar cada tipo de error como clave y su cantidad de apariciones como valor para analizar registros de una aplicación.
Configuración de aplicaciones: asociar nombres de parámetros con valores, como idioma con es.
```

Figura 3.1: Prompt a un LLM.

Tres aplicaciones de HashMap

1. **Consulta de alumnos.** Se asocia el número de cuenta con el registro del alumno. Al consultar la cuenta 12345, se recuperan los datos asociados; al actualizar esa misma clave, se reemplaza el registro anterior.
2. **Conteo de palabras.** Se asocia cada palabra con su frecuencia. Para la secuencia «sol, luna, sol», se registran dos apariciones de «sol» y una de «luna», sin exigir un orden de recorrido.
3. **Inventario por código.** Se vincula cada código de producto con sus existencias. Para el código A17 se pueden registrar 12 unidades y actualizar el valor a 11 después de una venta.

Tres aplicaciones de TreeSet

1. **Catálogo de nombres.** Se almacenan nombres sin duplicados y se recorren conforme al criterio de orden elegido. Al insertar «Ana», «Luis» y «Ana», se conservan únicamente «Ana» y «Luis».
2. **Consulta de fechas.** Se mantienen fechas distintas en orden cronológico. Para una fecha de referencia, se puede consultar la siguiente disponible o seleccionar las comprendidas en un intervalo.
3. **Clasificación de puntajes.** Se registran puntuaciones únicas y ordenadas. Al incorporar 80, 95 y 80, se conservan 80 y 95. Si se requiere distinguir participantes con igual puntaje, se debe incluir un identificador adicional en el criterio de comparación.

3.2. Organización de la aplicación

Se revisó el archivo proporcionado y se ejecutó su contenido sin modificarlo, mediante el modo de ejecución de archivos fuente de Java 17. En el método principal se crea el lector, se solicita un valor validado, se invoca la comprobación y se muestra el mensaje correspondiente. Finalmente, se cierra el lector. En la Figura 3.2 se resume esta secuencia; en las Figuras 3.3, 3.4 y 3.5 se detallan los subprocesos.

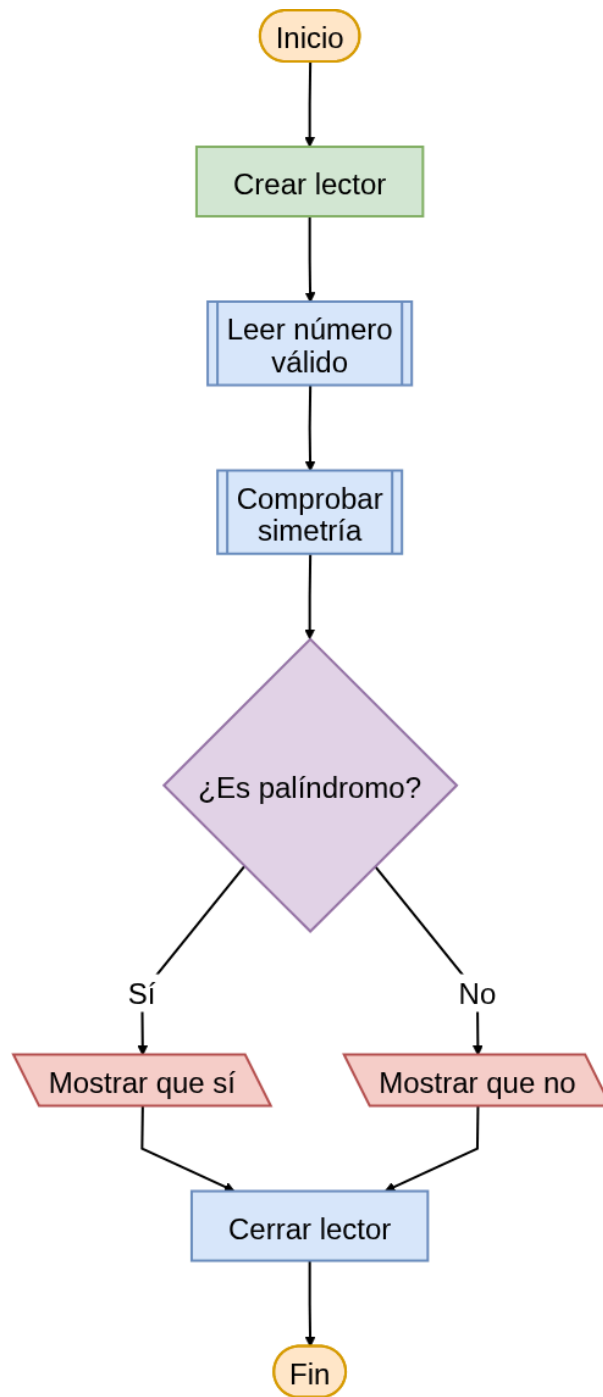


Figura 3.2: Secuencia general de la aplicación.

3.3. Lectura y comprobación del intervalo

En el método leerNumero se inicializa el valor en cero y se realiza al menos un intento de lectura. Primero se comprueba si el dato puede interpretarse como entero. Cuando se detecta una entrada inválida, se muestra el mensaje correspondiente y se descarta el dato. Si se lee un entero fuera del intervalo, se informa que no tiene cinco dígitos. En ambos casos se repite la solicitud. Solo se devuelve el número después de satisfacer la condición del intervalo, conforme a la Figura 3.3.

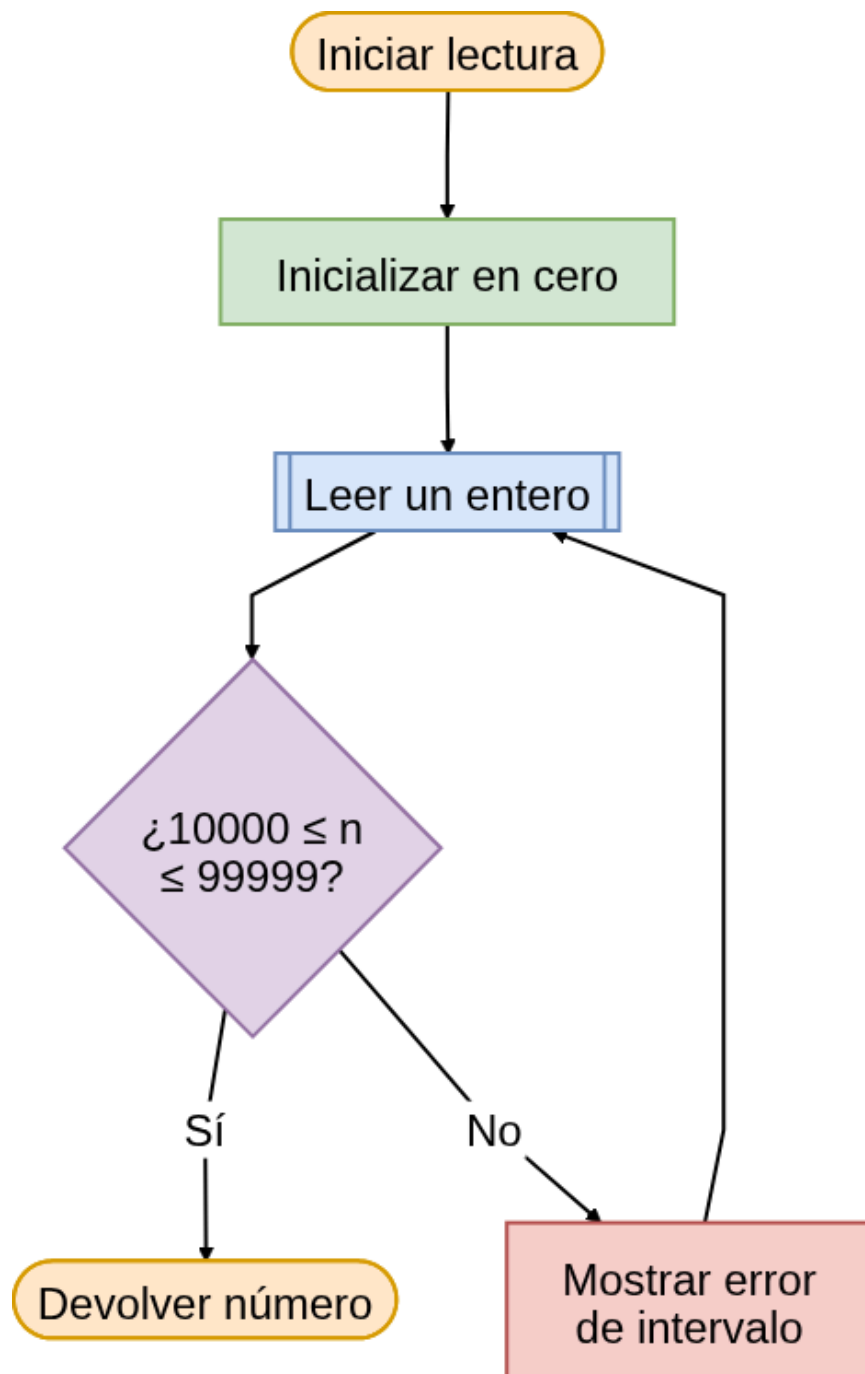


Figura 3.3: Repetición de la lectura hasta cumplir el intervalo.

En la Figura 3.4 se amplía la lectura de un entero indicada como subproceso en la Figura 3.3. Se separó esta parte únicamente para facilitar la lectura del diagrama; en el código se mantiene dentro de leerNumero, sin definir otro método. Se considera entero admisible el dato reconocido por el lector como representable por int.

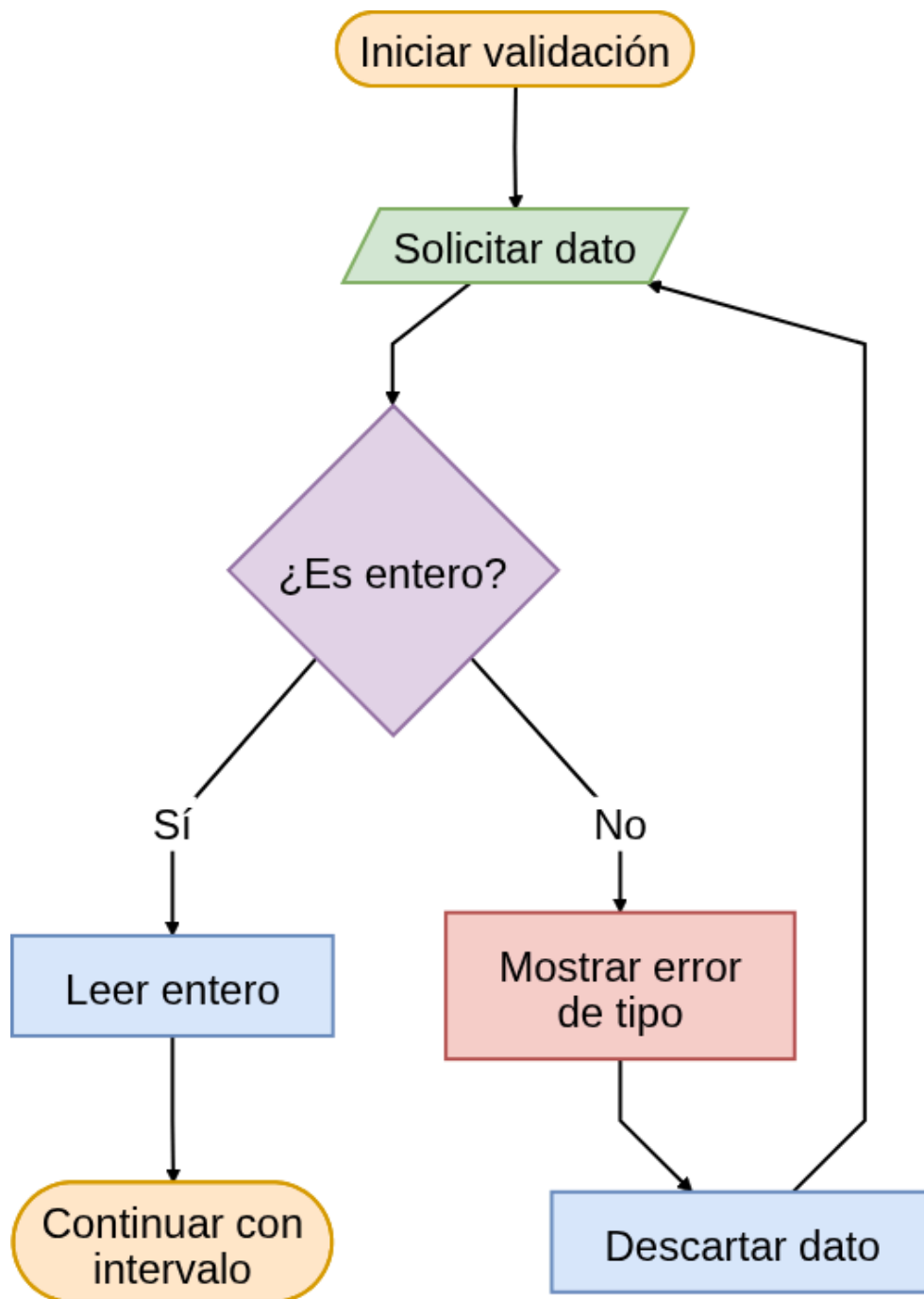


Figura 3.4: Validación del tipo dentro de la lectura del número.

3.4. Comparación de posiciones simétricas

En el método `esPalindromo` se extraen los cinco dígitos mediante división entera y residuo. Aunque en el archivo se calcula el tercero, no se utiliza en la decisión. Se devuelve un resultado verdadero únicamente cuando se cumplen ambas igualdades entre posiciones simétricas; en cualquier otro caso se devuelve falso. Con 12321 se comparan 1 con 1 y 2 con 2; con 12341 se satisface la primera igualdad, pero se rechaza la segunda. Se representa este procedimiento en la Figura 3.5.

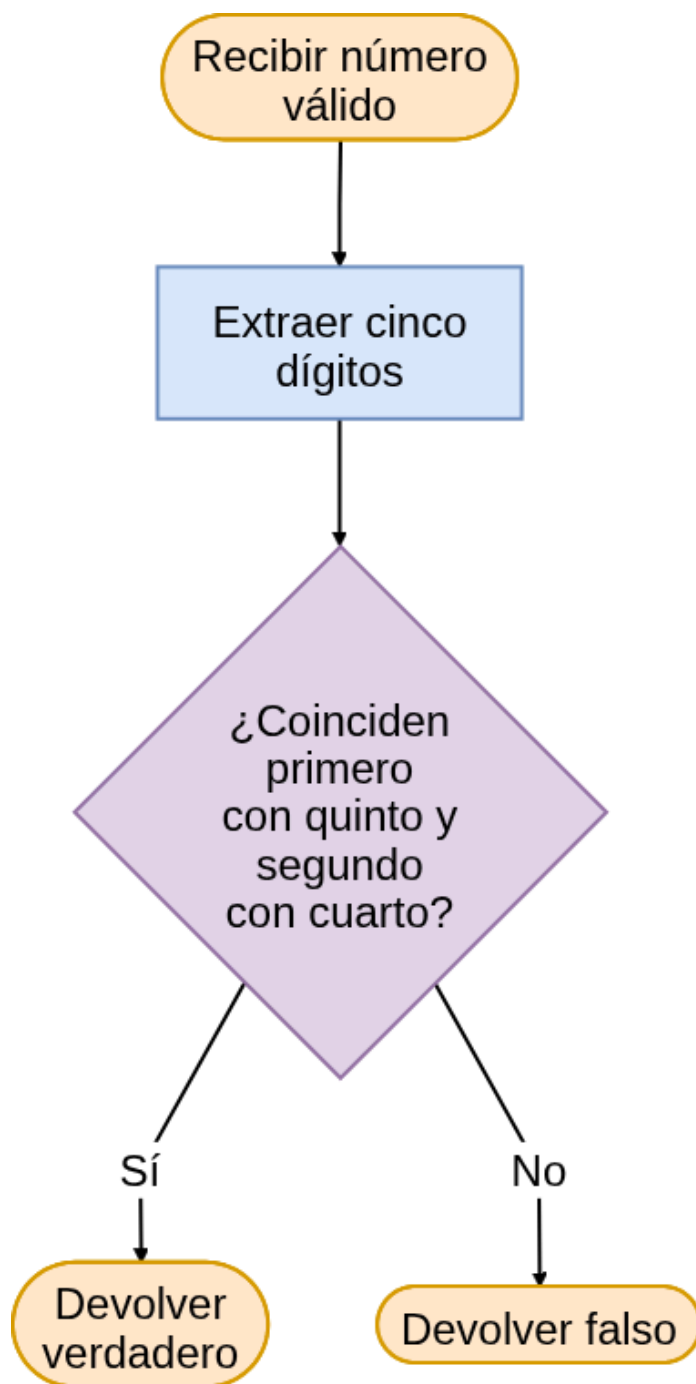


Figura 3.5: Comprobación de las posiciones simétricas.

4. Resultados

4.1. Clasificación de entradas válidas

En las nueve ejecuciones con un único dato válido se verificaron las clasificaciones de la Tabla 1. En todos los casos se mostró el número seguido de «es un palíndromo.» o «no es un palíndromo.», según correspondiera. Se reservan las Figuras 4.1 y ?? para incorporar las capturas de estas ejecuciones.

Tabla 1: Clasificaciones verificadas para enteros de cinco dígitos.

Entrada	Resultado verificado
12321, 55555, 45554, 11611	Se identificaron como palíndromos.
12345	Se identificó como no palíndromo.
12341	Se rechazó por diferencia entre segundo y cuarto dígitos.
10000	Se aceptó el límite inferior y se clasificó como no palíndromo.
99999	Se aceptó el máximo permitido y se clasificó como palíndromo.
10001	Se reconoció como palíndromo con ceros internos.

```
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 12345
12345 no es un palíndromo.
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 12341
12341 no es un palíndromo.
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 10000
10000 no es un palíndromo.
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 99999
99999 es un palíndromo.
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 10001
10001 es un palíndromo.
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 11611
11611 es un palíndromo.
```

Figura 4.1: Clasificación de los cuatro ejemplos del enunciado.

4.2. Rechazo de valores fuera del intervalo

En una misma ejecución se ingresaron, en orden, 9999, 100000, -12321, 0 y 12321. Para las primeras cuatro entradas se mostró «Error: el número ingresado no tiene cinco dígitos.» y se volvió a solicitar el dato. Con 12321 se terminó la lectura y se confirmó el palíndromo. En la Figura 4.2 se reserva el espacio de la captura correspondiente.

```
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: -12321
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: 1000000
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: 5974331
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: -12345
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: -11611
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: 10900000
Error: el número ingresado no tiene cinco dígitos.
Ingresa un número entero de cinco dígitos: 12321
12321 es un palíndromo.
```

Figura 4.2: Errores de intervalo y recuperación de la lectura.

4.3. Rechazo de datos no admitidos como int

Se ingresaron sucesivamente «hola», «12.5», «2147483648» y «12321». Ante los primeros tres datos se mostró «Error: debes ingresar un número entero.» y se permitió otro intento. Se descartaron las entradas rechazadas y, al recibir 12321, se emitió el resultado correcto. Para documentar la secuencia se reserva la Figura 4.3. No se debe interpretar el mensaje genérico como prueba de que 2147483648 carece de valor entero: se rechaza por exceder el máximo representable por int.


```
PS C:\Users\hecto\Downloads> javac Palindromo.java
PS C:\Users\hecto\Downloads> java Palindromo
Ingresa un número entero de cinco dígitos: 12.95
Error: debes ingresar un número entero.
Ingresa un número entero de cinco dígitos: 122.1
Error: debes ingresar un número entero.
Ingresa un número entero de cinco dígitos: 12385.12
Error: debes ingresar un número entero.
Ingresa un número entero de cinco dígitos: 135674.34
Error: debes ingresar un número entero.
Ingresa un número entero de cinco dígitos: -12312341.123
Error: debes ingresar un número entero.
Ingresa un número entero de cinco dígitos: 16761
16761 es un palíndromo.
PS C:\Users\hecto\Downloads> |
```

Figura 4.3: Errores de tipo y recuperación de la lectura.

5. Conclusión

Mediante la distinción entre asociaciones por clave y conjuntos ordenados se sustentó la selección de colecciones para los seis ejemplos planteados. Para consultas de registros, conteos e inventarios se justificó el uso de HashMap; para nombres, fechas y puntajes únicos se justificó el de TreeSet. De esta manera se atendió el objetivo de explicar el funcionamiento de ambas estructuras y relacionarlo con necesidades concretas, considerando sus reglas de unicidad, orden y costo de operación.

Con las pruebas realizadas se verificó el cumplimiento de los objetivos de lectura validada y reconocimiento de palíndromos de cinco dígitos: se rechazaron los datos incorrectos, se permitió su corrección y se clasificaron los casos admitidos mediante las dos comparaciones necesarias.

Al separar estas responsabilidades en métodos estáticos, se mantuvo una secuencia de procesamiento clara y se facilitó la revisión de sus condiciones. Se sustentó así la solución en operaciones enteras y control de flujo, dentro del alcance interactivo establecido para las pruebas.

6. Referencias

- [1] Oracle, «Clase HashMap», *Documentación de la API de Java SE 21*, 2023, título traducido al español. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html>. [Consultado: 8 de septiembre de 2026].
- [2] Oracle, «Clase TreeSet», *Documentación de la API de Java SE 21*, 2023, título traducido al español. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeSet.html>. [Consultado: 8 de septiembre de 2026].
- [3] Oracle, «Expresiones», *Especificación del lenguaje Java*, edición Java SE 21, cap. 15, secs. 15.17.2 y 15.17.3, 2023, título traducido al español. [En línea]. Disponible en: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html>. [Consultado: 8 de septiembre de 2026].
- [4] Oracle, «Clase Scanner», *Documentación de la API de Java SE 21*, 2023, título traducido al español. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html>. [Consultado: 8 de septiembre de 2026].
- [5] Oracle, «Clases», *Especificación del lenguaje Java*, edición Java SE 21, cap. 8, sec. 8.4.3.2, 2023, título traducido al español. [En línea]. Disponible en: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-8.html>. [Consultado: 8 de septiembre de 2026].